

Decode GEMM/MoE kernels (SILOTIGER-668)

 [SILOTIGER-667] MoE decode warp-decode kernels (small-M): FP8 + MXFP4 gate_up/down Created: 24/Jun/26

Updated: 03/Aug/26

Status:	Opened
Project:	Silo Tiger
Components:	FlyDSL, Kernels
Affects versions:	None
Fix versions:	None
Parent:	Decode GEMM/MoE kernels

Type:	Story	Priority:	Undefined
Reporter:	Remes, Sami	Assignee:	Aario, Sami
Resolution:	Unresolved	Votes:	0
Labels:	None		
Remaining Estimate:	Not Specified		
Time Spent:	Not Specified		
Original estimate:	Not Specified		

Attachments:	 gemm_decode_design.md
Issue links:	Relates relates to SILOTIGER-500 Decode GEMM/MoE kernels Rejected
Severity:	Medium
Epic Link:	Decode GEMM/MoE kernels
Sprint:	FlyDSL Sprint 4, FlyDSL Sprint 2, FlyDSL Sprint 3
Team:	

Description

Target

Specialized **decode-time** MoE MLP kernels for **very small M (number of tokens B = 1 to 4)** — single-token decode plus a few speculative/draft tokens. This is the regime where a dense MFMA tiling is mostly padding (a 16x16 tile is >=75% empty at M<=4), so the design uses **one wave (64 lanes) per output scalar** with v_dot2_f32_bf16 instead of the matrix core. Kernels are correct for larger B too, but the design and dispatch are tuned for B<=4; above the B=2-4 crossover a fused MFMA single-launch kernel (AITER ASM, see comparison below) starts to win.

Target HW: AMD Instinct MI350X/MI355X (**gfx950**, CDNA4), wave64.

Datatypes targeted

- **Activations (X / intermediate):** BF16 (decode default) and FP8.
- **gate/up weights:** FP8 (fast dot2/pk-f paths). MXFP4 only on the slow scalar path so far — fast FP4 gate_up is the top remaining work item.
- **down weights:** FP8 and **MXFP4 (packed FP4, two E2M1 per byte)** — fast path implemented.
- **Accumulation:** FP32. **Output/intermediate:** BF16.
- **Scales:** PerTensor, PerToken, Block2D (FP8 act B1ock2D<1,128>, FP8 weight B1ock2D<128,128>, MXFP4 weight B1ock2D<1,32> with e8m0 microscale).

Status / results (gfx950, per-kernel C++ bench)

MXFP4 down via the 2-outputs/wave (H2) layout now **beats the best FP8 `down`** (down time in ms, lower is better):

shape	B	fp8 best (down_h2_d2) ms	down_fp4_h2 ms	speedup
DeepSeek-V3	2	0.0323	0.0266	1.21x
DeepSeek-V3	4	0.0763	0.0515	1.48x
DeepSeek-V3	8	0.1543	0.1267	1.22x

DeepSeek-V3	32	0.6114	0.4714	1.30x
Qwen3Next (INTER=512)	4	0.0131	0.0086	1.52x
Qwen3Next (INTER=512)	8	0.0178	0.0155	1.15x

Neutral at B=1 (use the wide 1-output FP4 variant there). 0 register spills, 0 scratch, 8 waves/SIMD (max occupancy). Correct across 4 scale layouts x {1-output, H2}
vs CPU reference.

Design (reimplementation-grade summary)

Two-stage split, BF16 intermediate in HBM between launches: gate_up_fused (x, fp8/bf16 W) -> inter[B,TOPK,INTER] BF16 -> down_reduce (fp8/fp4 W) -> y[B,HIDDEN]. The intermediate is tiny at decode B, and both kernels are **memory-bound on weight reads** (gate_up 90%, down ~74-79% of achievable HBM), so the only ~2x lever is **fewer weight bytes (FP4)**, not compute or cache tricks. Collapsing to one launch (persistent or atomic-merge) was tried and loses below the B=4 crossover.

gate_up: grid B*TOPK*INTER waves; each wave computes one inter[token,slot,neuron] = silu(gate.x)*(up.x). Loop HIDDEN in 64*kVector tiles; lane l owns [1*kVector,(l+1)*kVector); convert FP8 W (and FP8 X) to BF16 pairs via v_cvt_scalef32_pk_bf16_fp8; v_dot2_f32_bf16 accumulate gate & up; apply x_scale_w_scale per K-block (Block2D scales broadcast through LDS); butterfly-reduce 64 lanes; lane 0 writes silu(gate)_up.

down_reduce: grid B*ceil(HIDDEN/HPerWarp) waves; each wave owns 1 (HPerWarp=2: two) output channel(s) and sums over TOPK then INTER. FP8 -> v_cvt_scalef32_pk_bf16_fp8 + dot2. **MXFP4 -> raw packed 128-bit load (memcpy to avoid strict-aliasing UB) + `v\_cvt\_scalef32\_pk\_bf16\_fp4` + s_nop-free `v\_dot2` with 4 (H2: 8) independent FP32 accumulators + one drain (`s\_nop 2`) before summing.** MX block scale applied after the dot (block_k=32 covers the lane chunk). Accumulate router_wt*scale; butterfly-reduce; lane 0 writes y. **H2 (2 outputs/wave)** doubles in-flight weight loads and reuses the BF16 activation — the key win.

Instruction selection (key)

- v_dot2_f32_bf16 — BF16.BF16->FP32 dot (2 MAC/lane); dependent chain needs s_nop 2 (or independent accumulators + one drain).
- v_cvt_scalef32_pk_bf16_fp8 / ..._fp4 — packed FP8/FP4 -> BF16 with scale.
- v_cvt_scalef32_pk_f32_fp4, v_cvt_pk_f32_fp8, v_pk_fma_f32 — alternative FP32 path (slower for down; activation is BF16).
- butterfly XOR warp_shuffle x6 for the 64-lane reduce.

AITER fmoe ASM comparison (incl. the flat sorting-free kernel)

(1) Kernel-time profile vs the real ASM all-fused, *sorting-free kernel* aiter::fmoe_bf16_blockscaleFp8_g1u1_flat_vs_ps_silu_1x128 vs the split warp-decode pair (gate_up+down), same shapes/routing/process, gfx950 (docs/issues/warp_decode_profiling/2026-06-04-asm-fmoe-profiling.md). The ASM kernel is MFMA-only on 1/16-filled decode-M tiles and never memory-stalled, but **occupancy-starved**:

shape	B	ASM fmoe us	split (gate_up+down) us	winner	ASM occ%	ASM HBM%
Qwen3Next	1	16.3	14.6	split +12%	1.2	24
Qwen3Next	8	50.9	63.9	ASM -20%	6.9	61
DeepSeek-V3	1	85.6	72.9	split +17%	5.3	52
DeepSeek-V3	8	450.4	513.0	ASM -12%	11.7	77

Mechanism: at low B the ASM grid (~40 waves at Qwen B=1) leaves the machine 88-99% empty so the many-small-CTA split pair wins; at high B the ASM kernel fills up, nears the HBM roofline, and avoids the split's BF16 round-trip, so it wins. down is the chronic weak point on every shape (16/34/55/61% of peak HBM).

(2) End-to-end MoE block (topk+quant+GEMM) vs aiter.fused_moe per-1x128 FP8 blockscale, best of {default heuristic, exhaustive tune}

, MI350X (docs/reviews/warp-decode-bench-results.md, 2026-04-21). Tuned AITER dispatches to the 1-stage ASM flat kernels (fmoe_bf16_blockscaleFp8_g1u1_vs_pf2_silu_16x128, ..._vs_silu_1tg_32x256). Total us, lower=better:

shape	B	AITER (best) us	WD-FP8 us	WD-BF16 us	winner
DeepSeek-V3	1	88.0	101.7	93.6	AITER 1.16x
DeepSeek-V3	8	421.1	584.6	577.5	AITER 1.39x
MiniMax	1	40.7	40.8	36.4	WD-BF16 1.21x
MiniMax	8	153.2	208.9	196.0	AITER 1.36x

Warp-decode wins only **B=1 on MiniMax (WD-BF16)**; ITER wins B=1 DeepSeek and all B>=2 (1.2-3.2x) via moe_sorting weight reuse + single launch.

Both comparisons predate the MXFP4 down + H2 work in this branch (which narrows the down gap at B>=2); a refreshed head-to-head with down_fp4_h2 (and FP4 gate_up once built) is the natural next measurement.

Tuning knobs

- **kVector** 8/16/32 (load width; 16 = one 128-bit FP8 transaction; 8 for FP4 fast path; 32 = wide single-transaction FP4).
- **kHPerWarp (down)** 1/2 — **2 is the best** at B>=2.
- **kUseDot2** vs **kUsePackedFp32**; **kNPerWarp (gate_up)** 1/2; **kWarpsPerBlock** (LDS staging); **kLanesPerOutput** (short-INTER subgroup).

SplitK lever (portable from the GEMM decode work)

The down stage is the chronic low-BW kernel (16-61% of peak) and at small grids (Qwen short-INTER, low B) it is **occupancy-bound**, not at the HBM wall. The gemm_decode kernels added a **cross-block split-K** (k_batch): a second grid axis where each shard does a partial dot over a K-slice, with an atomicAdd (non-deterministic) or scratch-reduce (deterministic / batch-invariant) epilogue. It triggers only when `grid * k_batch <= CuCount` (the under-occupied regime) — exactly the down/Qwen-B1 case. The same axis maps onto down (split INTER) and gate_up (split HIDDEN). The atomic path needs a **zeroed output**, which can be folded into the gate_up epilogue / a prologue rather than a standalone fill kernel — the same trick as the vLLM `blockscale_splitk_zero_init` fusion that made split-K free for the blockscale GEMM. Caveat: in GEMM decode the k_batch sweep was null on the **compute-bound** M>=5 leg; decode down is memory/occupancy-bound at small grids (where split-K should pay) but this is **untested for MoE**. ITER's fmoe also uses `{{ksplit in {2,3}}`

`}}` for occupancy at very small B — same lever. Refs: docs/gemm_decode_design.md section 10; docs/reviews/vllm-zero-init-splitk-review.md.

XCD swizzle: conflicting GEMM-vs-MoE data (needs a re-test)

The skinny-GEMM gemm_decode work found XCD/chiplet workgroup remapping to be a **large win at small N** (M=1/N=2048: **7.59 TB/s = 95% HBM vs 4.09 TB/s = 51%** for wvSpltk). The MoE warp-decode sweeps found it **neutral/regressing**. This is a real conflict to revisit, not a settled result. Most-likely reasons (hypotheses):

- **XCD remap co-locates cross-wave data reuse in L2, and MoE decode has almost none.** GEMM decode reuses a shared operand across its mp/np register tile, so remapping turns that into L2 locality. MoE warp-decode is one wave per output scalar with no register tile — each wave streams a distinct expert weight row and discards it, so there is nothing to co-locate.
- **Expert collision is rare at decode.** The only MoE reuse is two tokens routed to the same expert; expected reuse is $\sim B \cdot \text{TOPK} / E \sim 1.1x$ at the ref shapes (DeepSeek E=256, Qwen E=512), so even a perfect token->XCD remap finds nothing to share. (This matches the intuition that it only helps when multiple tokens hit the same expert, which is unlikely in decode.)
- **Tiling / grid / batch differ.** GEMM's win was at small-N grids that under-fill 256 CUs (remap also rebalances occupancy); the MoE sweeps were dominated by large-grid DeepSeek (already full) and the packing variant regressed. The small-grid MoE analogue (Qwen INTER=512, B=1) was **not** isolated, so it isn't apples-to-apples.
- **Action:** re-test XCD swizzle on MoE specifically on small-grid Qwen (INTER=512/256/128, B=1) and again after any cross-wave reuse tiling (token-batched kMPerWarp / LDS staging) lands. Until then, treat 'XCD swizzle doesn't help MoE decode' as **regime-limited, not refuted**.

Remaining / untried (priority order)

1. **FP4 gate_up** (apply the FP4 down recipe; the biggest remaining ~2x decode win; gate on accuracy).
2. **B=1 FP4 down software-pipelined prefetch** (currently ~2.96 TB/s, MLP-bound).
3. **Cross-block split-K on down** (see SplitK lever above) + zero-init fusion.
4. **Re-test XCD swizzle on MoE** (small-grid Qwen; after a reuse tiling lands).
5. Fuse input-side BF16->FP8 quant into gate_up; 8-warp double-buffered LDS down; kVector autotuning.

Tried and dropped (caveats: several are shape/regime-specific, not definitive)

- Persistent producer-owned single-stage (V2/V3/V4): 3.5-4x slower (grid-barrier + ~256-CTA cap).
- LDS gate/up X-staging: helps DeepSeek-BF16 only. LDS down intermediate: slower at 4-warp reuse (8-warp retest untried).
- NPerWarp=2 (gate_up): mixed/slight loss (halves occupancy; weights dominate).
- XCD swizzle: neutral on large-grid DeepSeek (conflicts with GEMM decode — see section above).
- Short-INTER wide/subgroup down: slower on Qwen. down_fp4_h2_wide: dominated. MFMA-fp8 into split: refuted (already at memory roofline). Weight-reuse register tile: not feasible at decode (reuse ceiling ~1.1x at ref shapes).

- Caveat: HBM-% figures depend on a gfx950 counter correction (FetchSize/TCC_MISS undercount HBM 2x; cross-check the EA counter).

Code / reference

- Branch (pushed, no PR): users/samremes/ck/warp-decode on ROCm/rocm-libraries, commit 62e30c9098.
- Full reimplementaion doc (in branch): projects/composablekernel/include/ck_tile/ops/warp_decode/WARP_DECODE_MOE_KERNELS.md (XCD reconciliation in section 11.1).
- Kernels: projects/composablekernel/include/ck_tile/ops/warp_decode/. Tests/bench: projects/composablekernel/test/ck_tile/warp_decode/.

Part of the Decode GEMM/MoE epic ([🔗 SILOTIGER-500](#) REJECTED).

Comments

Comment by [Remes, Sami](#) [24/Jun/26]

MoE decode child of the Decode GEMM/MoE umbrella ([🔗 SILOTIGER-500](#) REJECTED). Convert 500 to an Epic via the UI Move wizard, then this can be re-linked as a proper Epic child.

Comment by [Remes, Sami](#) [28/Jul/26]

Notes from the publicly released K3 technical report (analysis, not measured data)

Read the serving/kernel section (§5.4.2, pp. 24–25) of the publicly released K3 technical report. Everything below is read off that public document — there are no measurements of our kernels in it, so it informs design direction and priority only, and does not replace our own benchmarking.

Independent validation of the design in this ticket. The report describes routed-expert MoE decode at small batch as "memory-bound streaming of weight matrices — a regime for which conventional tile-centric kernels are poorly suited due to their compute-oriented design and preprocessing overheads" (p.25), and builds its MoE decoding kernel on a token-centric design "in which each warp is responsible for one output neuron and streams the associated weights directly from memory" (p.25). That is the same parallelism axis and the same rejection of MFMA-tiled grouped GEMM at decode M that this ticket rests on, chosen for a production deployment at frontier scale (16 of 896 routed experts activated per token, pp. 1, 6) with MXFP4 expert weights and MXFP8 activations (p.14) — the same quantized weight-streaming target as the FP8 and MXFP4 work here. Their bibliography cites the same public warp-decode write-up our design derives from (their reference 12).

Two techniques in the report that we do not currently have:

- **Lane teams over disjoint expert subsets.** "To further increase parallelism, we subdivide each warp into finer-grained lane teams, each processing a disjoint subset of experts, followed by a warp-wide reduction of the partial results" (p.25). For the down stage this maps to partitioning wave64 into T teams of 64/T lanes, each team taking TOPK/T experts, shortening the serial TOPK loop by T. The epilogue is already compatible: the butterfly reduce yields the cross-team sum for free, and the router weight is already folded per expert into the accumulator. Cost: per-lane element count grows by T, so the vectorized-load divisibility constraint tightens to lanes-per-team times kVector dividing INTER (interacts with the kVector 8/16/32 selection and the short-INTER shapes), and the expert id and scale metadata stop being wave-uniform. Closest existing knob is kLanesPerOutput, but that subdivides the reduction axis, not the expert axis. Caveat: the report motivates this purely as "increase parallelism", which is the same lever as the split-K / occupancy item on the down stage — so it is most likely to pay in the small-grid, occupancy-bound regime, not where we are already near the achievable-HBM wall.
- **Offline weight permutation to reduce runtime dequantization.** "The weight layout is permuted offline at a one-time preprocessing cost, substantially reducing the runtime dequantization overhead" (p.25). That is the entire description — no granularity, instruction sequence, or numbers — so treat it as a direction, not a recipe. It targets a cost we have already measured to be first-order: the choice of packed FP8/FP4-to-BF16 scale-conversion instruction sequence was the difference between a regression and a win. The generic form is to pre-interleave the packed FP4/FP8 elements at model-load time so nibbles and bytes arrive in the order the packed conversion and lane mapping want, removing shift/mask/shuffle work from the inner loop, and to co-locate the microscale with the block it covers. Implementation implication: this is a weight-prepack step outside the kernel, so it needs a declared, versioned layout contract rather than the implicit expert-major weight-tensor assumption — cheapest to introduce alongside the FP4 gate/up item (top of the remaining list) rather than retrofitted afterwards. The same holds for any re-implementation of this kernel family at ISA or DSL level: both techniques are much cheaper to design in from the start than to bolt on.

Adjacent, outside this kernel but affecting its measured end-to-end benefit (same passage, p.25): shared-expert computation is used as the overlap partner for the latent all-gather rather than being serialized, and the latent down-projection is fused with the MoE router into a single GEMM with the output all-gather fused into the GEMM epilogue. Noted as context for whoever wires this into a serving stack.

Fuller treatment, with page citations and with the places where the report is too thin to act on, is in the warp-decode design doc, section 9.4 ("External Corroboration from a Public Frontier-Scale Serving Report"):

```
docs/warp_decode_design.md
```